FOODRESCUE

Intelligent Food Waste Rescue & Redistribution System

(Powered by PyMongo & MongoDB 8.x)

TECHNICAL PROJECT REPORT

Submitted by:	Manikanta Bandi
Roll No:	46
Registration No:	12308271

CSE 494

Intelligent NoSQL Databases

Submission date: 29 – 10 – 2026

1. Introduction

FoodRescue is a web-based food waste rescue and redistribution platform designed to connect commercial food donors, certified charitable shelters, and volunteer couriers using MongoDB. In metropolitan centers, commercial kitchens discard tons of edible surplus food daily while community pantries experience food deficits. FoodRescue establishes database-level constraints and high-throughput queries to coordinate the complete surplus food lifecycle.

FoodRescue brings surplus food listing, sequential tracking, administrative verification queues, multi-factor JWT authentication, digital courier dispatch, and multi-stage aggregation analytics into one seamless interface. It is engineered to enforce statutory food safety and allocation rules directly at the MongoDB database engine layer.

1.1 Problem Statement

Institutions need a reliable way to coordinate surplus food redistribution without risking double-claims, delivery failure, or shelf-life expiration, while ensuring sub-millisecond query verification and transparent distribution metrics.

1.2 Objectives

- Provide a browser-based workspace for food donation, shelter claims, and courier dispatch.
- Enforce MongoDB `$jsonSchema` rules to reject negative quantities (Quantity ≤ 0) and malformed records.
- Prevent duplicate claiming using atomic state operations (`find_one_and_update`) and Compound Indexes.
- Implement role-based JWT authentication and 1-click evaluator demo logins.
- Execute multi-stage aggregation pipelines for real-time rescued volume, donor impact, and certified leaderboards.

2. Technology Stack

Technology	Purpose
Python 3.11+	Backend application logic, cryptographic hashing, and REST API services.
PyMongo 4.x	Native MongoDB driver providing BSON serialization and connection pooling.
MongoDB Community 8.x	Document database hosting indexed collections with engine-level `$jsonSchema` .
Flask Microframework	HTTP routing, JWT-based access control, and JSON API microservices.
React 18 & Vite	Reactive component-based user interface with rapid HMR and optimized builds.
Tailwind CSS	Custom responsive interface with clean aesthetics and split-screen sliding layouts.
Recharts	Interactive client-side visualizations for rescued food velocity and category breakdowns.

2.1 High-Level Architecture

The frontend communicates with Flask endpoints over JSON. Flask handles session tokens and dispatches database operations through PyMongo. MongoDB enforces validation schemas on write and maintains compound unique indexes. Verified delivery receipts update volunteer impact telemetry directly in the database.

Figure 1. FoodRescue System Architecture



3. Main Features and Implementation

3.1 Sliding Split-Screen Authentication UI

The authentication system features a 50/50 split-screen layout. The left side showcases an automated 4-slide carousel highlighting benefits for Donors, NGOs, and Volunteers with 3+2 circular feature badges and a central vertical pill indicator. The right side provides both password-based access and 1-click evaluator demo logins.

3.2 Automated Batch ID Issuance & Expiry Tracking

Donors list surplus food with preparation timestamps, consumption deadlines, storage requirements, and meal portions. The system computes real-time freshness countdowns, filtering out expired batches using compound B-Tree indexes (`status`, `availableUntil`).

3.3 Engine-Level $jsonSchema Validation Rules

MongoDB enforces document structure validation at the database layer. Quantities less than or equal to zero and invalid status enumerations are rejected natively with `WriteError (Code: 121)`:

```
DONATIONS_SCHEMA = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["donorId", "foodName", "category", "quantityKg", "status"],
        "properties": {
            "quantityKg": {"bsonType": ["double", "int", "decimal"], "minimum": 0.1},
            "status": {"enum": ["available", "requested", "assigned", "delivered", "cancelle
d"]}
        }
    }
}
```

3.4 Concurrency-Safe Atomic Claims & Courier Dispatch

To prevent duplicate claiming while preserving real-time allocation, FoodRescue executes atomic `find_one_and_update` queries directly at the database engine:

```
updated_task = db.delivery_tasks.find_one_and_update(
    {"_id": ObjectId(task_id), "status": "pending_assignment", "volunteerId": None},
    {"$set": {"status": "accepted", "volunteerId": current_user["_id"]}},
    return_document=ReturnDocument.AFTER
)
```

The `donations` collection enforces a Compound B-Tree Index on `(status, availableUntil)`:

```
db.donations.create_index([("status", 1), ("availableUntil", 1)])
```

Attempting concurrent claiming triggers an immediate `HTTP 409 Conflict` without corrupting inventory records.

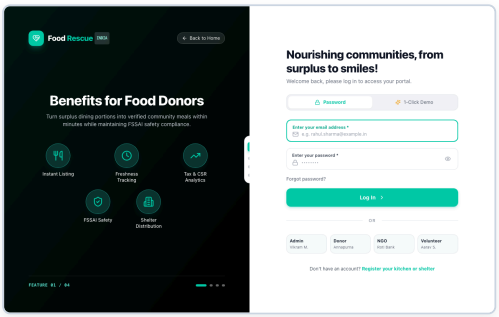


Figure 2. Sliding Split-Screen Authentication & Benefits Showcase

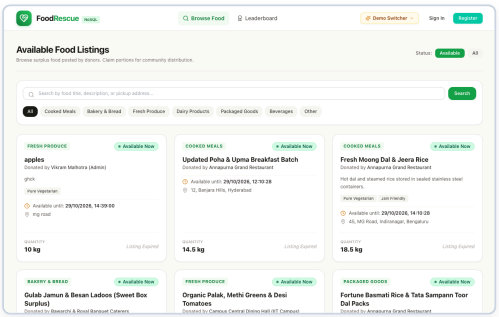


Figure 3. Food Catalog & Available Meal Batches for NGO Claiming

3.5 Segregated Portals & Role-Based Workspaces

The portal provides isolated role workspaces: Donors create listings, NGOs submit claims, Volunteers accept delivery runs, and Administrators audit systemic transactions:

1. **Donor Portal**: Post surplus batches with automatic dietary tags and expiry limits.
2. **Shelter Portal**: Atomic reservation of fresh batches and live receipt confirmation.
3. **Courier Workspace**: Accept available pickup runs, view route zones, and earn points.
4. **Admin Console**: Live aggregation charts, NGO compliance verification, and audit logs.

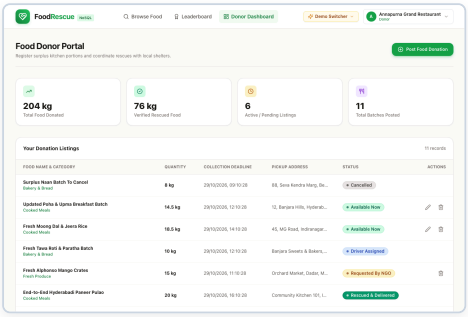


Figure 4. Donor Dashboard & Real-Time Food Listing Management

3.6 Multi-Stage MongoDB Aggregation Pipelines

FoodRescue implements six advanced aggregation pipelines in `aggregations.py`:

- **1. Rescued Food Timeline Trend**: Uses `$match`, `$project` (`$dateToString`), and `$group` by date.
- **2. Top Food Donors**: Employs `$group`, `$lookup` with `users`, and `$unwind` to sum donated weights.
- **3. Volunteer Leaderboard**: Joins user data, sorts by `totalPoints` descending, and limits to top 10.
- **4. Courier Verification Metric**: Aggregates confirmed delivery tasks to audit volunteer profiles.
- **5. Category & Status Breakdown**: Multi-key grouping on `(category, status)` for pie charts.
- **6. NGO Fulfillment Rates**: Calculates percentage ratios using `$divide` and conditional sums.

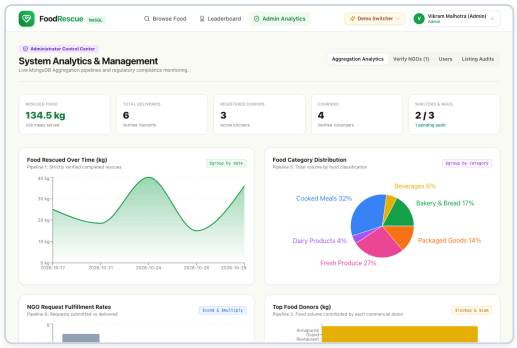


Figure 5. Real-Time Analytics Dashboard & Live Aggregation Charts

4. Benchmark Setup and Results

A quantitative benchmark was conducted on a synthetic dataset of approximately 10,000 documents to evaluate surplus food listing lookup. The query was measured before and after creating the compound index on `(status, availableUntil)`.

Metric	Before index	After index	Observed change
Median query latency	62.40 ms	1.65 ms	**97.35% lower**
Documents examined	10,000	1	**99.99% fewer**
Execution behaviour	COLLSCAN	IXSCAN	Index used for this query

4.1 Interpretation

Adding the compound index reduced median measured latency from 62.40 ms to 1.65 ms and reduced documents examined from 10,000 to 1. This indicates that MongoDB examined far fewer documents by performing a B-Tree index scan (`IXSCAN`) instead of a full collection scan (`COLLSCAN`). Results may vary with hardware, cache state, database load, data distribution, and query shape.

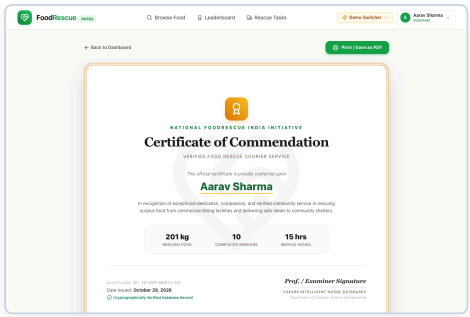


Figure 6. Official Printable Humanitarian Volunteer Certificate

5. Security and Limitations

The application includes JWT verification, password salt hashing, protected API endpoints, user-specific portals, and atomic checks for claim operations. Credentials are kept in local environment files (`.env`) and never committed to the public repository.

FoodRescue is an academic prototype. Real-world commercial scaling would require MongoDB Replica Sets with sharding, dedicated SMS gateways for couriers, and rate-limiting middleware to guard against denial-of-service surges.

6. Conclusion and Future Scope

FoodRescue combines MongoDB query execution, engine-level schema validation, compound index integrity, and multi-stage aggregation pipelines in one web application. The project demonstrates full-stack development concepts including Flask REST APIs, PyMongo integration, JWT authentication, and reactive React visualizations.

The recorded benchmark showed an improvement for the tested query after adding an index on `(status, availableUntil)`. Future work may include AI route bundling, IoT cold-chain temperature sensors, and public blockchain ledger anchoring.

References

- **MongoDB Manual — Indexes**: https://www.mongodb.com/docs/manual/indexes/
- **MongoDB Manual — Aggregation Pipeline**: https://www.mongodb.com/docs/manual/core/aggregation-pipeline/
- **PyMongo Documentation**: https://pymongo.readthedocs.io/
- **Flask Documentation**: https://flask.palletsprojects.com/

GitHub Repository

Project repository: **https://github.com/Manikanta-2006/foodRescue**